

運用 Artifact Registry 管理依附元件

來源：<https://codelabs.developers.google.com/dependency-management-artifact-registry?hl=zh-tw>

步驟數：5

瞭解如何透過 Architect Registry 管理依附元件。

目錄

1. [1. 總覽](#)
2. [2. 設定和需求](#)
3. [3. 使用容器映像檔](#)
4. [4. 使用語言套件](#)
5. [5. 恭喜！](#)

步驟 1 · 約 0 分鐘

1. 總覽

[Artifact Registry](#) 由 Container Registry 演變而來，可讓組織在同一個位置管理容器映像檔和語言套件 (例如 Maven 和 npm)。這項服務與其他 Google Cloud 工具和執行階段完全整合，並支援以語言為基礎的依附元件管理，可搭配 npm 和 Maven 等工具使用。因此您可輕鬆整合 CI/CD 工具，並設定自動化管道。

本實驗室將逐步介紹 Artifact Registry 的部分功能。

學習目標

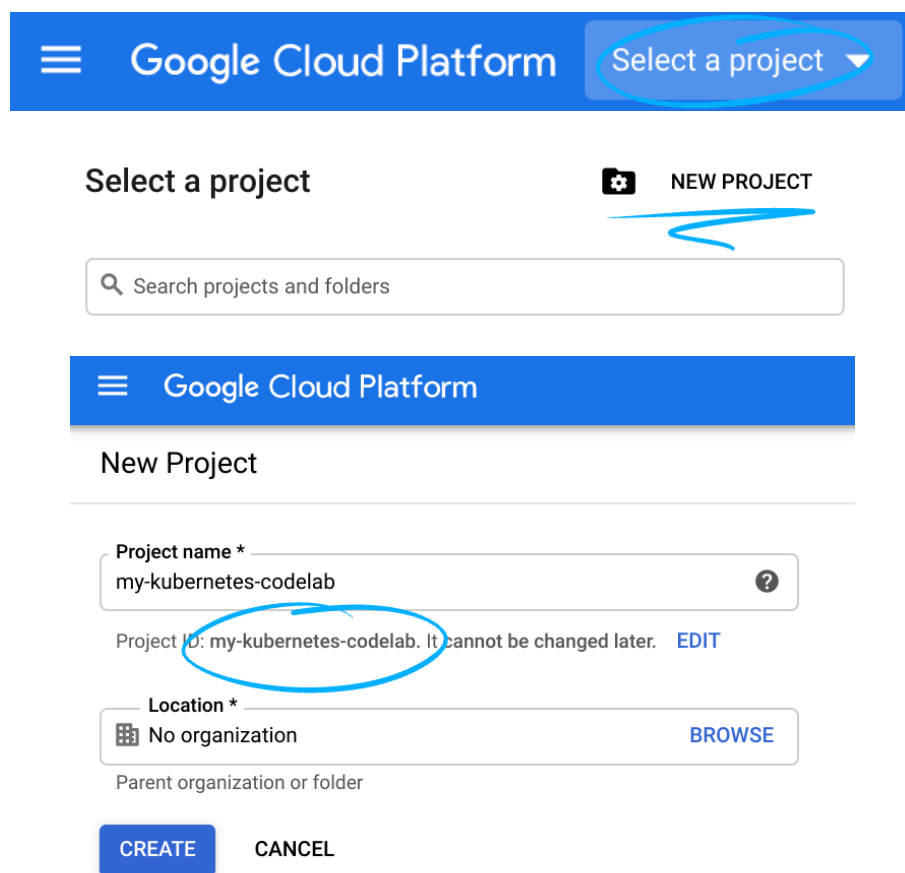
這個實驗室的學習目標為何？

- 為容器和語言套件建立存放區
 - 使用 Artifact Registry 管理容器映像檔
 - 設定 Maven 來使用 Artifact Registry 取得 Java 依附元件
-

2. 設定和需求

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。



- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新該位置資訊。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生不重複的字串，通常您不需要在意這個字串。在大多數程式碼研究室中，您需要參照專案 ID (通常會標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間都會維持這個設定。
- 請注意，部分 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是專屬 ID，選定後就無法變更，且其他使用者無法使用。您是該 ID 的唯一使用者。即使刪除專案，該 ID 也無法再使用

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成本程式碼研究室的費用應該不高，甚至完全免費。如要關閉資源，避免產生本教學課程以外的費用，您可以刪除自己建立的資源，或刪除整個專案。Google

Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

設定 gcloud

在 Cloud Shell 設定專案 ID 和專案編號，分別儲存為 `PROJECT_ID` 和 `PROJECT_NUMBER` 變數。

〇〇〇

```
export PROJECT_ID=$(gcloud config get-value project)
export PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --format='value(projectNumber)')
```

啟用 Google 服務

〇〇〇

```
gcloud services enable \
  cloudresourcemanager.googleapis.com \
  container.googleapis.com \
  artifactregistry.googleapis.com \
  containerregistry.googleapis.com \
  containerscanning.googleapis.com
```

取得原始碼

本實驗室的原始碼位於 GitHub 的 GoogleCloudPlatform 組織。使用下列指令複製，然後切換至該目錄。

```
git clone https://github.com/GoogleCloudPlatform/cloud-code-samples/
```

步驟 3 · 約 0 分鐘

3. 使用容器映像檔

在 Artifact Registry 建立 Docker 存放區

Artifact Registry 支援管理容器映像檔和語言套件。不同構件類型適用的規格不同，舉例來說，Maven 依附元件和 Node 依附元件的要求就不一樣。

為支援不同的 API 規格，Artifact Registry 需要知道您希望 API 回應採用哪種格式。為此，您需要建立存放區，並傳入 `--repository-format` 旗標，指出所需的存放區類型

在 Cloud Shell 執行下列指令，為 Docker 映像檔建立存放區：

```
gcloud artifacts repositories create container-dev-repo --repository-format=docker \
--location=us-central1 --description="Docker repository for Container Dev Workshop"
```

如果出現 Cloud Shell 授權提示，請點選「授權」。

前往 Google Cloud 控制台的「Artifact Registry - Repositories」，就會看到新建立的 Docker 存放區 `container-dev-repo`。點選後，會發現存放區目前是空的。

設定 Docker 對 Artifact Registry 的驗證機制

連線至 Artifact Registry 時，必須提供憑證才能授予存取權。Docker 可設定為直接使用 gcloud 憑證，不必另外設定憑證。

在 Cloud Shell 執行下列指令，將 Docker 設為使用 Google Cloud CLI，驗證對位在 `us-central1` 區域的 Artifact Registry 傳送的要求：

```
gcloud auth configure-docker us-central1-docker.pkg.dev
```

系統會提示您確認是否要變更 Cloud Shell Docker 設定，請按下 Enter 鍵。

探索範例應用程式

您在先前的步驟中複製的 Git 存放區提供範例應用程式。切換至 java 目錄並查看應用程式程式碼。

```
cd cloud-code-samples/java/java-hello-world
```

這個資料夾包含一個範例 Java 應用程式，可呈現簡單的網頁。除了與本實驗室無關的各種檔案外，還包含 `src` 資料夾中的原始碼，以及您將用來在本機建構容器映像檔的 Dockerfile。

建構容器映像檔

如要將容器映像檔儲存至 Artifact Registry，必須先建立一個。

執行下列指令，建構容器映像檔並加上正確標記，以便在下一個步驟中推送至存放區：

```
docker build -t us-central1-docker.pkg.dev/$PROJECT_ID/container-dev-repo/java-hello-world:tag1 .
```

將容器映像檔推送至 Artifact Registry：

執行下列指令，將容器映像檔推送至先前建立的存放區：

```
docker push us-central1-docker.pkg.dev/$PROJECT_ID/container-dev-repo/java-hello-world:tag1
```

在 Artifact Registry 查看映像檔

前往 [Google Cloud 控制台 - Artifact Registry - 存放區](#)，點按 `container-dev-repo`，確認是否有 `java-hello-world` 映像檔。按一下圖片，注意標有 `tag1` 的圖片。您會看到安全漏洞掃描正在執行或已完成，以及偵測到的安全漏洞數量。

Artifact Registry

Repositories

Settings

Digests for java-hello-world

DELETE

SETUP INSTRUCTIONS

us-central1-docker.pkg.dev

>

galloro-demos

>

container-dev-repo

>

java-hello-world

Filter

Enter property name or value

	Name	Description	Tags	Created	Updated	Vulnerabilities
	161d4b213681		tag1	Just now	Just now	42

點按安全漏洞數量，即可查看映像檔中偵測到的安全漏洞清單，包括 CVE 公告名稱和嚴重程度。點按各項安全漏洞旁的「查看」，即可查看詳細資料：

CVE-2021-46848

Fixed in	4.16.0 2+deb11u1
Documentation	
Debian	CVE-2021-46848
Details	
Version	4.16.0 2
Affected location	cpe:/o:debian:debian_linux:11
Package	libtasn1-6
Package type	OS
Long description	NIST vectors: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:H
CVSS V3 score	9.1
Create time	Feb 3, 2023
Update time	Feb 3, 2023
Occurrence name	projects/qwiklabs-gcp-01-0689e0256165/occurrences/9bf5fd02-d7b5-4a16-9173-1f3c596813f3
Note name	projects/goog-vulnz/notes/CVE-2021-46848
Provider name	goog-vulnz

SHOW LESS INFO

步驟 4 · 約 0 分鐘

4. 使用語言套件

在本節中，您將瞭解如何設定 Artifact Registry Java 存放區並上傳套件，然後在不同應用程式中運用這些套件。

建立 Java 套件存放區

在 Cloud Shell 執行下列指令，為 Java 構件建立存放區：

```
gcloud artifacts repositories create container-dev-java-repo \
  --repository-format=maven \
  --location=us-central1 \
  --description="Java package repository for Container Dev Workshop"
```

如果出現 Cloud Shell 授權提示，請點選「授權」。

前往 Google Cloud 控制台的「Artifact Registry」-「Repositories」，就會看到新建立的 Maven 存放區 `container-dev-java-repo`。點選後，會發現存放區目前是空的。

設定 Artifact Repository 的驗證機制

使用下列指令，以使用者帳戶憑證更新應用程式預設憑證 (ADC) 的已知位置，這樣 Artifact Registry 憑證輔助程式就能在連線至存放區時，使用這些憑證驗證：

```
gcloud auth login --update-adc
```

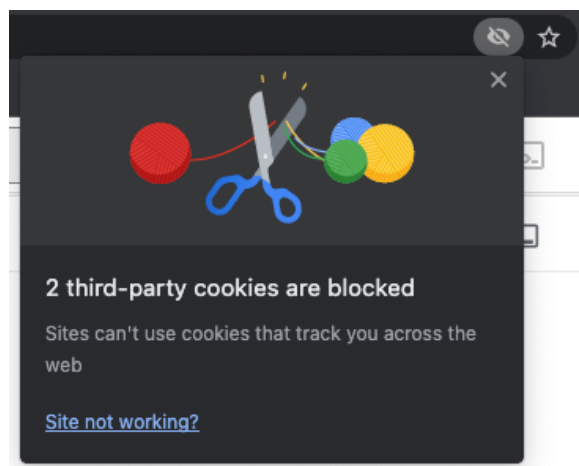
設定 Maven 以搭配 Artifact Registry 使用

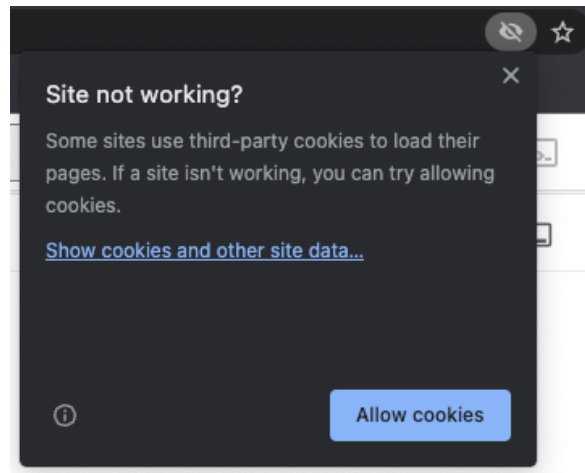
在 `java-hello-world` 資料夾執行下列指令，開啟 Cloud Shell 編輯器，並將應用程式資料夾新增至這個工作區：

```
cloudshell workspace .
```

按一下「網站現在可以正常運作嗎？」然後按一下「允許 Cookie」，啟用第三方 Cookie。

瀏覽器重新載入後，開啟 Cloud Shell 並再次執行上述指令，載入應用程式資料夾。





在 Cloud Shell 編輯器中開啟 pom.xml，然後按一下「Open Editor」（開啟編輯器）

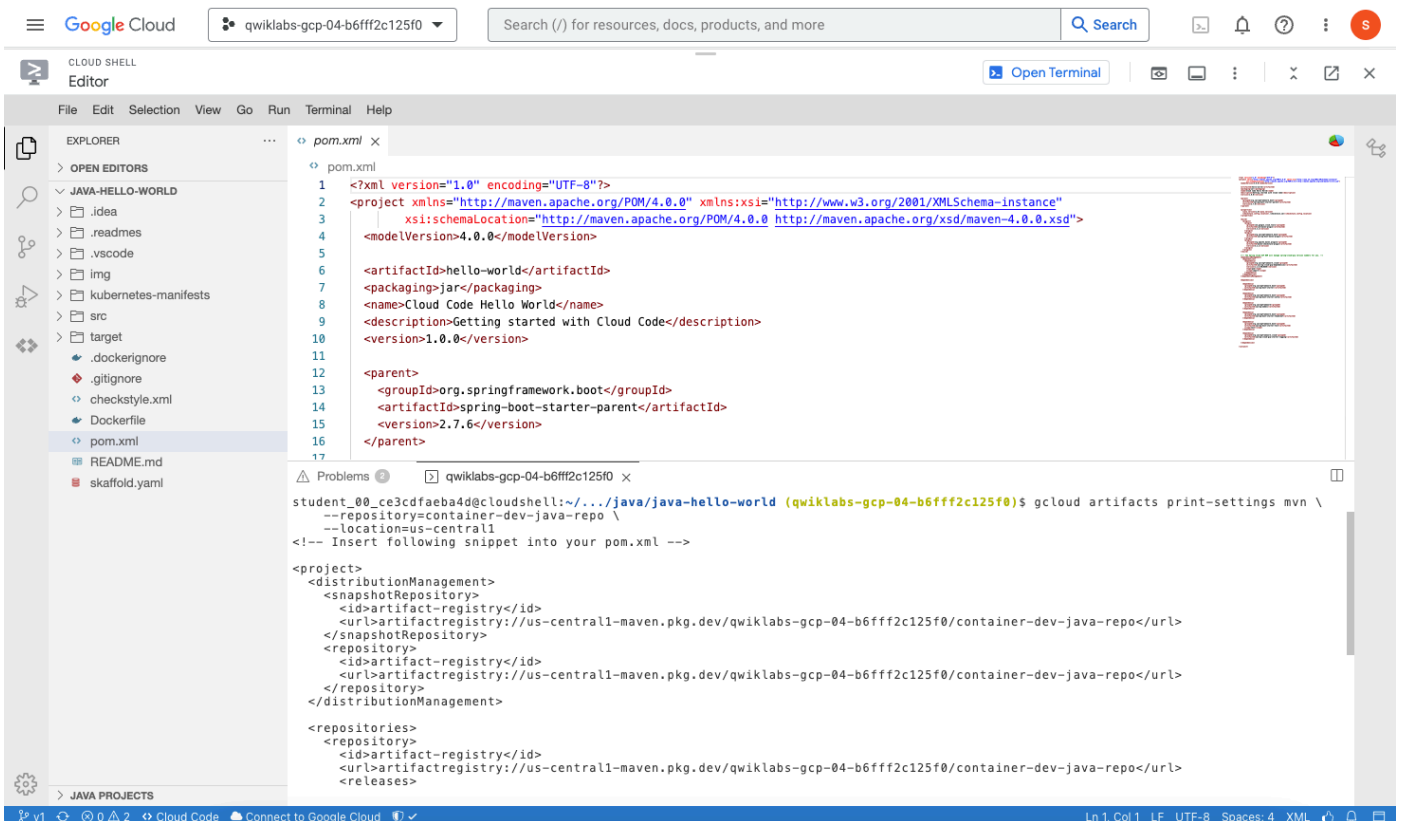


從 Cloud Shell 編輯器開啟終端機，然後執行下列指令，顯示要新增至 Java 專案的存放區設定：

```
gcloud artifacts print-settings mvn \  
  --repository=container-dev-java-repo \  
  --location=us-central1
```

，然後將傳回的設定新增至 pom.xml 檔案中的對應區段。

內建終端機的 Cloud 編輯器檢視畫面：



更新 **distributionManagement** 區段

```

<distributionManagement>
  <snapshotRepository>
    <id>artifact-registry</id>
    <url>artifactregistry://us-centrall1-maven.pkg.dev/<PROJECT>/container-dev-java-repo</url>
  </snapshotRepository>
  <repository>
    <id>artifact-registry</id>
    <url>artifactregistry://us-centrall1-maven.pkg.dev/<PROJECT>/container-dev-java-repo</url>
  </repository>
</distributionManagement>

```

更新 **repositories** 區段

```

<repositories>
  <repository>
    <id>artifact-registry</id>
    <url>artifactregistry://us-centrall1-maven.pkg.dev/<PROJECT>/container-dev-java-repo</url>
    <releases>
      <enabled>true</enabled>
    </releases>
    <snapshots>
      <enabled>true</enabled>
    </snapshots>
  </repository>
</repositories>

```

更新擴充功能

```

<extensions>
  <extension>
    <groupId>com.google.cloud.artifactregistry</groupId>
    <artifactId>artifactregistry-maven-wagon</artifactId>
    <version>2.1.0</version>
  </extension>
</extensions>

```

您可以參考以下完整檔案的範例。請務必將 **<PROJECT>** 替換為您的專案 ID。

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <artifactId>hello-world</artifactId>
  <packaging>jar</packaging>
  <name>Cloud Code Hello World</name>
  <description>Getting started with Cloud Code</description>
  <version>1.0.0</version>
  <distributionManagement>
    <snapshotRepository>
      <id>artifact-registry</id>
      <url>artifactregistry://us-centrall-maven.pkg.dev/<PROJECT>/container-dev-java-
repo</url>
    </snapshotRepository>
    <repository>
      <id>artifact-registry</id>
      <url>artifactregistry://us-centrall-maven.pkg.dev/<PROJECT>/container-dev-java-
repo</url>
    </repository>
  </distributionManagement>

  <repositories>
    <repository>
      <id>artifact-registry</id>
      <url>artifactregistry://us-centrall-maven.pkg.dev/<PROJECT>/container-dev-java-
repo</url>
      <releases>
        <enabled>true</enabled>
      </releases>
      <snapshots>
        <enabled>true</enabled>
      </snapshots>
    </repository>
  </repositories>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.6.3</version>
  </parent>

  <properties>
    <java.version>1.8</java.version>
    <checkstyle.config.location>./checkstyle.xml</checkstyle.config.location>
  </properties>

  <build>
    <plugins>
```

```

<plugin>
  <groupId>com.google.cloud.tools</groupId>
  <artifactId>jib-maven-plugin</artifactId>
  <version>3.2.0</version>
</plugin>
<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-checkstyle-plugin</artifactId>
  <version>3.1.2</version>
</plugin>
</plugins>
<extensions>
  <extension>
    <groupId>com.google.cloud.artifactregistry</groupId>
    <artifactId>artifactregistry-maven-wagon</artifactId>
    <version>2.1.0</version>
  </extension>
</extensions>
</build>

<!-- The Spring Cloud GCP BOM will manage spring-cloud-gcp version numbers for you. -->
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-gcp-dependencies</artifactId>
      <version>1.2.8.RELEASE</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

<dependencies>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jetty</artifactId>
  </dependency>

  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-webmvc</artifactId>

```

```
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>

<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-gcp-starter-logging</artifactId>
</dependency>

</dependencies>

</project>
```

將 Java 套件上傳至 Artifact Registry

在 Maven 設定 Artifact Registry 後，即可使用 Artifact Registry 儲存 Java JAR，供貴機構的其他專案使用。

執行下列指令，將 Java 套件上傳至 Artifact Registry：

```
mvn deploy
```

在 Artifact Registry 檢查 Java 套件

前往 [Cloud 控制台 - Artifact Registry - 存放區](#)，然後點按 `container-dev-java-repo`，確認 `hello-world` 二進位檔構件已存在：

galloro-demos > us-central1 > container-dev-java-repo

Details

Version PolicyNone

Filter Enter property name or value

☐	Name ↑	Created	Updated
☐	 org.springframework.boot:hello-world	6 minutes ago	6 minutes ago

步驟 5 · 約 0 分鐘

5. 恭喜！

恭喜，您已完成本程式碼研究室！

涵蓋內容

- 為容器和語言套件建立存放區
- 使用 Artifact Registry 管理容器映像檔
- 設定 Maven 來使用 Artifact Registry 取得 Java 依附元件

清除所用資源

執行下列指令即可刪除專案：

```
gcloud projects delete $PROJECT_ID
```
